TapTap Adaptive Game Engine

Checkpoint 3 — Prototype Build

S. S. Prajwala

Deadline: March 22, 2026

4 Game Types | Leaderboard | Admin Panel

1. Project Overview

TapTap is a fully configurable, plugin-based adaptive learning game engine built with React, TypeScript, and Vite. Every aspect of gameplay — questions, levels, scoring rules, difficulty progression, and UI behavior — is driven entirely by JSON configuration files. No game logic is hardcoded in any component.

Project Name	TapTap Adaptive Game Engine
Stack	React 18, TypeScript (strict), Vite, CSS Variables
Architecture	Plugin-based engine with JSON-driven config
Game Types	Quiz, Pattern Puzzle, Flashcard, Memory Match
Checkpoint	3 — Prototype Build
Deadline	March 22, 2026

2. What Was Built in Checkpoint 3

2.1 Four Playable Game Types

Each game type is a fully independent plugin with its own React component, type guard, and optional scoring override. Adding a new game type requires creating exactly two files — the plugin and a JSON config.

- Quiz (quiz) — Multiple choice questions with timer, streak multiplier, hints, and explanations

- Pattern Puzzle (puzzle) — Extends arithmetic, geometric, and quadratic sequences automatically
- Flashcard (flashcard) — Flip-card self-assessment with category tags and streak scoring
- Memory Match (memory) — Card-pair matching game with animated flip and move counter

2.2 Leaderboard System

Scores are persisted to localStorage via LeaderboardService. After every game, the player enters their name and saves their score. The leaderboard is filterable by game or global, shows rank, accuracy, and date, and supports clearing all entries.

2.3 Admin Panel — In-Browser Game Configuration

The Admin Panel is the key demonstration of JSON-driven gameplay. Instead of editing raw JSON files, game conductors can configure the entire game through a browser interface. All changes update the engine config in real time.

- Questions tab — Add, edit, and delete quiz questions using a form. You set the correct answer by clicking Mark Correct. Hints and explanations are optional.
- Levels tab — Assign questions to levels via dropdown. Remove questions from levels with one click.
- Settings tab — Edit title, description, scoring thresholds, streak multiplier, and UI toggles.

Note: In the current prototype, the Admin Panel is open to all users to demonstrate the JSON-driven engine capability. In the next checkpoint, access will be restricted to authenticated admins only — regular players will not be able to modify game content.

2.4 Polished UI

Complete visual overhaul with JetBrains Mono + Syne fonts, per-game accent colors on cards, top-border highlight on hover, dark theme throughout, and animated progress bars. Each game card shows its plugin type, emoji, level count, and question count.

3. Engine Architecture

3.1 Folder Structure

```
src/
├── types/
│   └── engine.types.ts        ← All TypeScript types (single source of truth)
├── engine/
│   ├── EngineCore.ts          ← Pure reducer — reduce(state, action): GameState
```

```
│   ├── ScoreEngine.ts          ← Points, streaks, time bonuses — config-driven
│   ├── AdaptiveEngine.ts       ← Evaluates JSON rules, selects questions
│   ├── LevelManager.ts         ← Level progression and unlock conditions
│   ├── PluginRegistry.ts       ← Singleton map: pluginId → GamePlugin
│   └── LeaderboardService.ts   ← localStorage persistence for scores
├── plugins/
│   ├── index.ts                ← Only file that registers plugins
│   ├── quiz/QuizPlugin.tsx
│   ├── puzzle/PuzzlePlugin.tsx
│   ├── flashcard/FlashcardPlugin.tsx
│   └── memory/MemoryPlugin.tsx
├── games/
│   ├── logic-game.json         ← 12 quiz questions, 3 levels
│   ├── pattern-puzzle.json     ← 8 patterns, arithmetic/geometric/quadratic
│   ├── world-capitals.json     ← 9 flashcard questions
│   └── emoji-memory.json       ← 3 memory levels with pairs
├── hooks/
│   └── useGameEngine.ts        ← Wires useReducer + timer + hints
├── components/
│   └── GameRenderer.tsx        ← Dynamic renderer, zero plugin coupling
├── pages/
│   ├── LeaderboardPage.tsx
│   └── AdminPanel.tsx
├── App.tsx                     ← Navigation only, zero game logic
└── styles.css
```

3.2 File Reference

File	Layer	Purpose
`engine.types.ts`	Types	Every TypeScript interface in the entire project
`EngineCore.ts`	Engine	Pure reducer for useReducer — no side-effects
`ScoreEngine.ts`	Engine	calculate(), updateStats(), buildFeedback()
`AdaptiveEngine.ts`	Engine	evaluate() rules from JSON, selectQuestions()
`LevelManager.ts`	Engine	Level progression, unlock conditions, reset()
`PluginRegistry.ts`	Engine	Singleton: register(), get(), list()
`LeaderboardService.ts`	Engine	save(), getGlobal(), getForGame(), getRank()
`plugins/index.ts`	Plugins	Only file that imports and registers plugins
`QuizPlugin.tsx`	Plugins	Multiple choice: Component + validateQuestion
`PuzzlePlugin.tsx`	Plugins	Sequence detection: arithmetic/geometric/quad
`FlashcardPlugin.tsx`	Plugins	Flip card + self-grade correct/incorrect
`MemoryPlugin.tsx`	Plugins	Card pair matching with shuffle and animation
`useGameEngine.ts`	Hook	useReducer + timer + hints + event subscription
`GameRenderer.tsx`	UI	Resolves plugin from registry, renders screens
`AdminPanel.tsx`	UI	Questions / Levels / Settings editor

File	Layer	Purpose
`LeaderboardPage.tsx`	UI	Filterable score table with localStorage
`App.tsx`	UI	Navigation only — library, leaderboard, admin

4. JSON-Driven Gameplay

Changing the JSON config changes the game. No code modification is needed for any of the following:

- Question content, options, correct answers, explanations, hints
- Number of levels and which questions appear in each level
- Unlock conditions between levels (minimum score required)
- Adaptive rules: accuracy thresholds that adjust difficulty automatically
- Scoring: base points, time bonus per second, streak threshold and multiplier
- UI flags: show/hide timer, progress bar, streak counter
- Which plugin renders the game (quiz, puzzle, flashcard, memory)

4.1 JSON Config Shape

```
{
  "id":    "logical-reasoning-v2",
  "title": "Logical Reasoning",
  "plugin": "quiz",              // ← changes the entire game type
  "questions": [
    {
      "id": "q-easy-1",
      "type": "quiz",
      "difficulty": "easy",
      "points": 100,
      "timeLimit": 60,
      "prompt": "2 + 2 = ?",
      "options": ["2","3","4","5"],
      "correctIndex": 2,         // ← you decide what is correct
      "hint": "Count your fingers",
      "explanation": "2+2=4"
    }
  ],
  "adaptiveRules": [
    {
      "condition": { "metric": "accuracy", "operator": ">=", "value": 0.8 },
      "action":    { "type": "adjustDifficulty", "payload": { "difficulty": "hard"
} }
    }
  ],
  "scoring": {
    "basePoints": 100,
    "timeBonus": true,
```

```
    "timeBonusPerSecond": 3,
    "streakMultiplier": true,
    "streakThreshold": 3,
    "streakMultiplierValue": 1.5
  }
}
```

5. Before vs After Refactor

Feature	Old System	New System
EngineCore	static run() one-shot call, no state	Pure reduce(state, action) for useReducer
AdaptiveEngine	decide() returned raw string 'NEXT'/'REPEAT'	evaluate() returns typed AdaptiveAction[]
ScoreEngine	hardcoded responseTime: 30, attempts: 1	All thresholds from ScoringConfig in JSON
QuizPlugin	start(level) { console.log() } — renders nothing	Typed GamePlugin<Q> with real Component
App.tsx	80+ lines of game logic, timer, level transitions	Zero game logic — config selector only
Question cycling	QuestionEngine static state, reset() manually	Pure selectQuestions() in AdaptiveEngine
Player state	PlayerState class mutated from handleAnswer()	Immutable stats updated in reducer
Analytics	AnalyticsEngine.logEvent() scattered in App.tsx	engine.on(listener) event bus
Leaderboard	Not present	LeaderboardService with localStorage
Admin Panel	Edit JSON files manually	In-browser form — no JSON editing needed
Game types	Quiz only	Quiz, Puzzle, Flashcard, Memory

6. How to Add a New Game Type

Adding a new game type requires exactly four steps and touches only two files:

1. **Add your question type to engine.types.ts** — extend the Question union type
2. **Create src/plugins/yourType/YourPlugin.tsx** — implement GamePlugin<YourQuestion>
3. **Register in src/plugins/index.ts** — one line: pluginRegistry.register(YourPlugin)

4. **Create src/games/your-game.json** — set "plugin": "yourType" and define questions

No other files need to change. GameRenderer resolves plugins via pluginRegistry.get(config.plugin) — it has zero knowledge of any specific game type.

7. Submission Details

Developer	S. S. Prajwala
GitHub Repository	https://github.com/SSPrajwala/taptap_adaptive_game_engine_deployed
Playable Link	https://taptapadaptivegameenginedeployed1.vercel.app/
Submission Portal	https://taptap.blackbucks.me/hackathon/6510/?testType=74
Deadline	March 22, 2026

8. Checkpoint 3 Criteria — How We Satisfy Each

Criterion	How It Is Satisfied
Playable game built using your engine	4 game types live on Vercel — Quiz, Puzzle, Flashcard, Memory. All fully playable with levels, scoring, and completion screens.
Gameplay fully controlled by JSON	Every question, level, adaptive rule, scoring config, and UI flag is in JSON. Engine reads config — no values are hardcoded anywhere.
Ability to change game behavior via config	Admin Panel lets you add/edit/delete questions, assign them to levels, and change scoring — all without touching any file. Changes take effect immediately.